VISOLD

VSD | BLOCKCHAIN PROTOCOL

A Modular Proof-of-Work Blockchain with Stake-Weighted BFT Finality, Resource-Aware Networking, and a Native Virtual Machine

TECHNICAL WHITEPAPER | VERSION 1.3

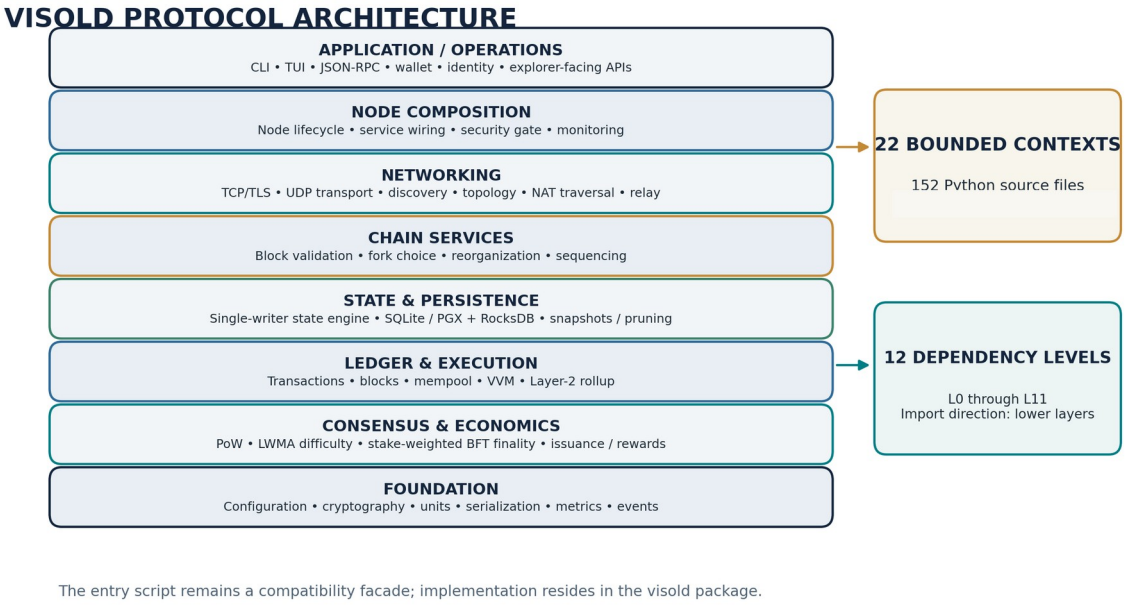


Figure 1. High-level architecture of the Visold software package.

Author Himadri Maity

Correspondence maity23himadri@gmail.com

Source code https://github.com/maity23himadri/Visold_blockchain

Protocol identity Network TESTNET | Software 11.0.0.0

Document date October 30, 2026

Document Control

Field	Value
Document	Visold (VSD) Technical Whitepaper
Version	1.3
Date	October 30, 2026
Software version	11.0.0.0
Repository-default CHAIN_ID literal	vsd-testnet-1 (active testnet repository default)
Author	Himadri Maity
Corresponding email	maity23himadri@gmail.com
Code repository	https://github.com/maity23himadri/Visold_blockchain
Source baseline	Public GitHub repository snapshot, pinned to commit 5f529cbaaf159792fa4ca261090edac534ee3f2b
Pinned public repository commit	5f529cbaaf159792fa4ca261090edac534ee3f2b
Package fingerprint	SHA-256: 68125018671909227b571088d1f91fe28492733911a0c09b63114af7f1309c64
Package inventory	152 Python files; 22 bounded contexts; 105 Python test modules
Software license	MIT License - docs/LICENSE.txt
Network status	Testnet (as of October 30, 2026; no mainnet launch)

Abstract

Visold (VSD) is a modular, account-based blockchain implementation written in Python and designed around a resource-aware node architecture. Its protocol combines proof-of-work (PoW) block production with stake-weighted Byzantine fault tolerant (BFT) finality when a sufficiently large validator set is available. During validator bootstrap, the chain uses deeper PoW confirmation finality; after BFT activation, validator signatures finalize blocks by a stake supermajority, with a PoW-depth liveness fallback when validator participation stalls. The implementation integrates a fractional-difficulty, linearly weighted moving-average (LWMA) adjustment; chain-domain-separated canonical transaction encoding; secp256k1 ECDSA; SHA-256 commitments; account-state execution; bounded peer-to-peer networking; rolling block-body pruning; authenticated state snapshots; a native smart-contract virtual machine; and an extensible Layer-2 rollup framework. The network described in this document is the Visold testnet; this whitepaper does not represent or announce a mainnet launch.

At the pinned repository snapshot, the `visold/` package contains 152 Python source files organized into 22 bounded contexts and 12 dependency levels; the repository also contains 105 Python test modules under `tests/`.

The document follows the code-defined protocol behavior of the public GitHub repository, pinned to commit 5f529cbaaf159792fa4ca261090edac534ee3f2b. Parameters are identified as consensus-critical, local policy, or deployment configuration wherever that distinction matters. In particular, local hashrate estimates and peer reports are kept outside consensus reward calculations; BFT votes use the validator set captured at the beginning of the relevant block; local anomaly freezes do not become consensus rules; and the Layer-2 proof interface distinguishes development authentication from a genuine validity-proof backend.

Keywords: blockchain protocol; proof of work; proof of stake; Byzantine fault tolerance; BFT finality; resource-aware networking; snapshot synchronization; Merkle commitment; virtual machine; Layer-2 rollup; consumer hardware; Python implementation.

Contents

Section	Topic
1	Protocol scope, identity and design objectives
2	Architecture and modular decomposition
3	Cryptographic primitives and identity
4	Transactions, blocks and account state
5	Proof-of-work mining and difficulty control
6	Stake-weighted BFT finality and validator safety
7	Monetary policy, fees and reward allocation
8	Hashrate governance and miner activity accounting
9	Peer-to-peer networking and topology management
10	Storage architecture, pruning and authenticated synchronization
11	Visold Virtual Machine (VVM)
12	Layer-2 rollup architecture
13	Mempool policy and commit-reveal transaction ordering
14	Self-healing, resilience and operational safeguards
15	Protocol governance and version upgrades
16	Wallet, identity, RPC and operator interfaces
17	End-to-end node lifecycle and protocol interactions
18	Verification architecture and multi-node test domains
19	Technical contributions and design significance
Appendix A	Consensus and deployment parameter reference
Appendix B	Data structures and message concepts
Appendix C	Source-code map
Appendix D	Glossary
References	Technical references and implementation sources

1. Protocol Scope, Identity and Design Objectives

1.1 Protocol identity

The native asset is VSD. The network described by this whitepaper is the Visold testnet; mainnet has not launched. At the pinned public repository snapshot (commit 5f529cbaaf159792fa4ca261090edac534ee3f2b), the source-default configuration is `Config.CHAIN_ID = "vsd-testnet-1"`. This is the active repository default for the testnet. The chain identifier is included in canonical L1 and L2 transaction signing data, transaction identifiers, receipt authentication and the VVM CHAINID domain, separating the testnet transaction domain from any future mainnet domain. A future mainnet deployment must use a distinct chain identifier and its own coordinated genesis identity. Software version and protocol version are distinct concepts: a software release can contain implementation changes that do not necessarily alter block validity, whereas a protocol-version transition can alter consensus interpretation and therefore requires coordinated activation.

Property	Configured value / behavior
Native unit	1 VSD = 100,000,000 satoshi
Repository-default CHAIN_ID literal	vsd-testnet-1 (see Section 1.1; network status is testnet)
Target block interval	60 seconds
Genesis timestamp	1,224,720,000 (2008-10-23 00:00:00 UTC)
Genesis subsidy	0 VSD
Genesis nonce	42
Initial difficulty	5.15
LWMA solve-time window	20 intervals after bootstrap
Block consensus size cap	4,096,000 bytes
P2P default port	8338 TCP and UDP
JSON-RPC default port	8339, loopback only
Relay port offset	2 from selected P2P port, when enabled
Network status	Testnet; mainnet has not launched

1.2 Design objectives

- Participation on ordinary devices: a node can run in a constrained environment, including Android through Termux, while keeping data directories, peer state, wallets, and network listeners isolated per instance.
- Consensus determinism: validity decisions derive from chain data and protocol constants rather than local mempool occupancy, peer-reported hashrate, local anomaly freezes, or operator-specific candidate construction policy.
- Layered fault containment: bounded contexts and directional dependency layers separate consensus, ledger, state, storage, networking, execution, and user interfaces.
- Explicit finality modes: the protocol has a PoW bootstrap path, stake-weighted BFT finality after validator-set activation, and a bounded PoW-depth fallback for liveness.
- Storage-aware synchronization: historical block bodies can be pruned while headers and authenticated state snapshots support new-node bootstrap and later continuation.
- Programmable execution: VVM smart contracts execute within explicit gas, stack, memory, call-depth, and bytecode bounds.
- Operational transparency: the node exposes RPC and terminal controls for chain status, peer state, governance, diagnostics, storage, wallet, contracts, and Layer-2 operations.

1.3 Scope and terminology

This whitepaper specifies the behavior represented by the referenced code snapshot. “Consensus rule” refers to a rule that every validating node must interpret consistently. “Local policy” refers to node-specific admission, scheduling, candidate selection, monitoring, or user-interface behavior that must not alter consensus validity. “Deployment configuration” refers to settings chosen by an operator, such as a storage backend, data directory, port plan, or external rollup proving backend. These categories are deliberately separated throughout the document.

2. Architecture and Modular Decomposition

2.1 Bounded contexts and dependency layers

The implementation is organized into 22 bounded contexts across dependency layers L0 through L11. The rule is directional: a module may import from its own bounded context or from a lower layer, but not from a higher layer. This prevents upward dependency cycles and makes the package structure auditable as a dependency graph. [4]

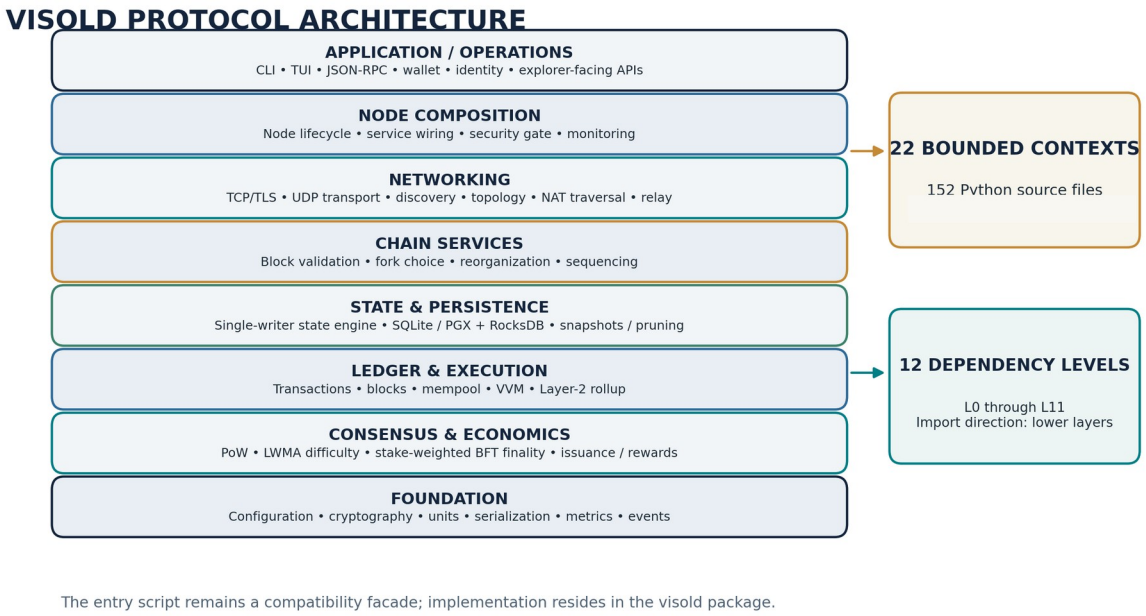


Figure 2. Layered package architecture and bounded-context inventory.

Layer	Bounded contexts	Primary responsibilities
L0	kernel	Configuration, compatibility helpers, clocks, metrics, logging, serialization, units, message identifiers, events and shared network helpers.
L1	crypto; economics; governance	Cryptographic primitives, reward and economic monitoring, version management and upgrade lifecycle.
L2	consensus; rollup; selfhealing	Difficulty and miner safety rules, Layer-2 state/proof framework, anomaly detection and response orchestration.
L3	resilience; vm; wallet	Circuit breaker, invariant and hardened-core checks, VVM runtime/tooling, wallet and HD key derivation.
L4	ledger	Transaction and block data models.
L5	state; storage	Single-writer state transition engine, persistence adapters, block database, snapshots and pruning.
L6	mempool	Pending transaction pool and commit-reveal policy.
L7	chain	Chain management, canonical block application, reorganization, consensus orchestration and rollup sequencing.
L8	network	P2P transport, sessions, TLS, peer management, discovery, topology policy, NAT traversal, relay and SPV.
L9	api; identity; mining; testing	JSON-RPC, name registry, mining workers and test/verification harnesses.

Layer	Bounded contexts	Primary responsibilities
L10	node	Node composition root, lifecycle, accounts and security gate.
L11	cli	Command-line interface, terminal UI and process entry point.

2.2 Source inventory and compatibility facade

The pinned repository contains 152 Python implementation files under `visold/`. The implementation is organized into 22 bounded contexts and a 12-level dependency graph. The entry file `visold_vsd_.py` is retained as the executable facade and compatibility namespace. The implementation is in the package, so module ownership and dependency boundaries remain visible without requiring operators to change the familiar entry command. The package build-identity record computes a package-source SHA-256 fingerprint over the source inventory; that fingerprint is a compatibility identifier, not a signed authorization manifest. [2, 3]

2.3 Node composition

`VisoldNode` composes the wallet, storage, blockchain, consensus engine, P2P services, RPC server, mining engine, governance, VVM, rollup sequencer, sentinel and self-healing components. The composition root supplies cross-context references late where needed: for example, the consensus engine receives its P2P network after network construction so validator votes can be gossiped without introducing a static import cycle. [17]

2.4 State ownership and write serialization

The state engine provides a single-writer path for canonical state changes. Incoming blocks, locally mined blocks and synchronization imports converge on the block-application path. This is important because state-root calculation, nonce advancement, balance updates, contract storage, role changes and reward credits must be observed in one deterministic order. Storage backends expose block-level atomicity, and the application path pairs database transactions with in-memory rollback markers so a failed transition does not leave a partially committed chain state. [5, 11]

3. Cryptographic Primitives and Identity

3.1 Hash functions and serialization

SHA-256 is the common digest primitive for transaction identifiers, block hashes, Merkle nodes, address derivation, selected protocol commitments and signature preimages. Structured block headers are hashed from a canonical JSON representation using sorted keys. Transaction signing uses a separate explicit binary encoding rather than relying on JSON serialization or ambiguous string concatenation.

3.2 secp256k1 ECDSA

Application keys use the secp256k1 elliptic curve. Transaction signatures are conventional ECDSA signatures represented internally as an `(r, s)` pair and serialized for network/state records. The mandatory implementation uses the `cryptography` package with OpenSSL. An optional Coincurve/libsecp256k1 acceleration backend is enabled only after a startup self-test that checks signing, valid verification, invalid-signature rejection and interoperability with the mandatory OpenSSL path. If the optional path fails its compatibility test, the implementation retains the mandatory backend.

The protocol preserves a historical double-SHA signature digest convention. The transaction layer first computes SHA-256 over the canonical transaction bytes; the ECDSA wrapper derives the effective digest by hashing that 32-byte message hash once more. This behavior is distinct from helper APIs designed to sign an already-finalized digest exactly once.

```
d_tx = SHA256( canonical_transaction_bytes )
d_ecdsa = SHA256( d_tx )
```

The public-key-to-sender binding is checked independently of signature validity. A valid signature by a key is not sufficient unless the address derived from the supplied public key equals the transaction sender. This prevents a transaction from naming an unrelated sender while presenting a valid signature from another key.

3.3 Address derivation

The public key is serialized into the protocol public-key byte representation. Visold computes SHA-256 over those bytes, computes SHA-256 again over the first digest, takes the first 20 bytes of the second digest, Base58-encodes that 20-byte value, and prefixes the printable address with `VSD`. Contract accounts use the same address namespace conventions. The protocol also recognizes explicit system addresses such as the burn address and bridge/withdrawal sentinels.

```
address_payload = SHA256(SHA256(public_key_bytes))[0:20]
```

The visible `VSD` prefix is a textual network/asset marker; it does not replace the cryptographic binding performed by the hashed payload. Address validation is therefore based on the expected string form and decoded payload, not merely on the prefix.

3.4 Wallet keystore and recovery phrase

Wallet key material is stored in an encrypted keystore. The configured keystore design uses AES-256-CBC with a password-derived key. Argon2id is used when the optional `argon2-cffi` dependency is available; the fallback is PBKDF2-HMAC-SHA256 with 260,000 iterations. The implementation includes deterministic derivation helpers and a 24-word recovery phrase representation. The phrase word list is project-specific and should be treated as Visold-native rather than presumed interoperable with standard BIP-39 wallets.

3.5 Verifiable random function

The codebase includes a secp256k1/SHA-256 try-and-increment VRF construction with deterministic nonce generation. During candidate preparation, the mining engine evaluates a VRF proof against the current tip hash and includes the proof and output in the candidate block header. The consensus module also contains a helper that ranks a supplied candidate set by VRF output. In the current block-production path, however, block acceptance is governed by the PoW target and normal block validation; the helper is not the active network-wide proposer-election protocol, and the current validator path does not make VRF verification a prerequisite for accepting a block. Accordingly, this paper treats VRF as a tip-bound header field and available primitive, not as the mechanism that currently replaces PoW proposer competition.

3.6 P2P transport identity

P2P sockets are wrapped in TLS. Nodes create a self-signed P-256 certificate, rotate it on a 90-day validity schedule, and use trust-on-first-use (TOFU) fingerprint pinning to detect unexpected certificate changes. TLS transport identity is separate from the secp256k1 application key used for transactions and validator votes. The RPC listener remains local-only by default and is not intended to be exposed directly to the public Internet. [7, 10, 23]

4. Transactions, Blocks and Account State

4.1 Account model

Visold uses an account-based state model. Canonical account state includes balances in satoshi, transaction nonces, contract code and storage, registered roles and stakes, name/identity claims, and additional protocol metadata. Transfers update sender and receiver balances; contract operations update VVM state; role-registration transactions update the role registry; rollup transactions commit Layer-2 batch transitions.

4.2 Transaction types and fields

Field / type	Protocol role
transfer	Move VSD between accounts.
deploy	Create a VVM contract and install its bytecode / initialization state.
call	Execute an existing VVM contract.
register	Register or modify a protocol role, perform stake-related registration/unstaking operations, or encode identity claims by memo convention.
rollup	Carry a Layer-2 rollup submission into Layer 1.
version	Transaction format version.
sender / receiver	Addresses that authorize and receive the state transition.

Field / type	Protocol role
amount / fee	VSD values canonicalized to integer satoshi for consensus calculations.
timestamp / expiry	Validity interval; default expiry is one hour after transaction construction, with an upper bound of 24 hours.
nonce	Per-account sequence value preventing replay and reordering within the account.
pubkey / signature	Public key and ECDSA authorization proof.
memo / data	Short human-readable memo and type-specific payload.
gas_limit / gas_price / contract_name	Execution budget and contract metadata for VVM operations.

4.3 Canonical transaction encoding

New transactions are encoded with the domain prefix `VSD-TX-V2\0`. Each field is serialized as a length-prefixed name and length-prefixed value; each name/value pair is itself length-prefixed before being appended to the output stream. The canonical field set binds the chain ID, transaction version, sender, receiver, amount in satoshi, fee in satoshi, timestamp, memo, nonce, expiry, transaction type, SHA-256 hash of data, gas limit, gas price in satoshi units and contract name. This removes field-boundary ambiguity and cross-chain replay between chain identifiers.

$$\texttt{TX_PREIMAGE} = \texttt{"VSD-TX-V2\0"} \;\|\|\; \Sigma(\; \texttt{len(item)} \;\|\|\; \texttt{item} \;)$$

The transaction identifier is SHA-256 over the canonical byte stream. Signature verification reconstructs exactly the same bytes and repeats the sender-key binding check. Legacy transaction formats are accepted only through an explicit historical-height compatibility gate; fresh transactions use the canonical encoding.

4.4 Fee and transaction validity

The normal transfer-fee rate is 1% of the transferred amount, computed in satoshi using integer arithmetic. The mempool also has a 1,000-satoshi minimum-fee admission floor for applicable transaction classes. That minimum is local admission policy, not a consensus requirement: a block containing an otherwise valid transaction is not invalid solely because a particular node would have refused to admit it to its own mempool. Transaction validity additionally checks finite numeric values, address form, signature, nonce, expiry, amount/fee relationships, type-specific payload structure and VVM gas constraints where relevant.

4.5 Block structure and commitments

A block contains a header and an ordered transaction body. The header fields committed by the block hash include block version, protocol version, height, previous-block hash, timestamp, miner address, difficulty, nonce, transaction Merkle root, resulting state root, VRF proof and VRF output. The block hash is produced from the canonical header dictionary. Validator vote records and the finalized flag are mutable post-mining metadata and are intentionally outside the hashed header, so asynchronous votes can accumulate without changing the PoW identity of the block.

Header field	Meaning
version / protocol_version	Block serialization version and protocol-rule version.
index	Height of the block in the canonical chain.
prev_hash	Hash pointer to the parent block.
timestamp	MTP-constrained block time.
miner	Address receiving proposer allocation and identifying the PoW producer.
difficulty / nonce	PoW search parameters.
merkle_root	Commitment to the ordered transactions.
state_root	Commitment to the resulting account / contract / role state.
vrf_proof / vrf_output	Tip-bound VRF data included in the current header format.

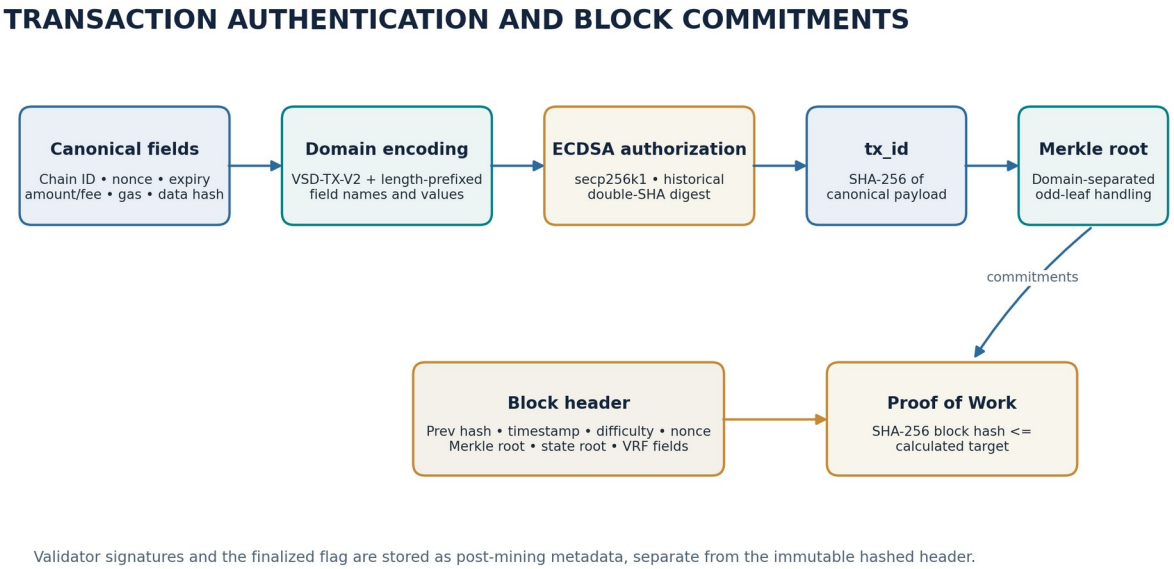


Figure 3. Canonical transaction authorization and block-level commitments.

4.6 Merkle tree construction

Leaf values are transaction identifiers. Parent nodes are SHA-256 over the concatenation of the two child hash strings. For an odd number of nodes at a level, the unpaired hash is promoted through a domain-separated hash using the literal marker `|odd|` rather than being duplicated and paired with itself. Empty transaction lists map to SHA-256 of the byte string `empty`. This distinct odd-leaf domain prevents the duplicate-tail ambiguity associated with historical duplicate-last Merkle constructions. Historical blocks below any configured activation boundary retain their historical Merkle interpretation; the default fresh-chain configuration applies the V2 rule from height zero.

4.7 Time and size validation

Block timestamps must be strictly greater than the median of the recent timestamp window, using up to the preceding 11 blocks, and no more than 7,200 seconds into the validating node's future. Candidate construction reads tip, difficulty and recent timestamps under the chain lock so that a reorganization cannot make the candidate combine parameters from different chain tips. The timestamp is held fixed through nonce search because it is part of the state-transition context and header commitment.

The consensus maximum block size is 4,096,000 bytes. Candidate construction uses a local advisory cap with a 1 MiB floor and dynamic adjustment: the cap grows by 10% when backlog conditions warrant and shrinks by 5% as the backlog clears. These candidate-building choices do not change the fixed validation maximum applied to incoming blocks.

4.8 State root and conservation

Block application executes transactions in canonical order, checks account nonces and spendable balances, executes contract state transitions, applies reward distribution and role effects, and computes the resulting state root. Consensus validation checks the incoming block's declared state root against the deterministic transition result. Monetary arithmetic is converted to integer satoshi. A supply-conservation invariant compares the sum of balances and protocol accounting against cumulative issued subsidy; transaction fees are transfers of existing balances and therefore are not counted as new issuance. Amounts assigned to the burn address remain accounted for as balances at that address. [5]

5. Proof-of-Work Mining and Difficulty Control

5.1 PoW target relation

The miner searches for a nonce such that the 256-bit integer represented by the block hash is no greater than the target derived from the block difficulty. Difficulty is fractional and expressed in hexadecimal-zero units. The target relation is approximately `2^(256 - 4D)` with an integer target construction that snaps the fractional component to a fixed grid and uses an integer-generated lookup table. The validation predicate is numerical, not a string-prefix comparison.

```
valid_PoW(H, D)  ⇔  0 <= int(H, 16) <= T(D)
```

$$T(D) \approx 2^{(256 - 4D)}, \text{ with fractional } D \text{ snapped to a } 1/4096 \text{ grid}$$

The target converter handles invalid or non-finite difficulty inputs by rejecting the attempted conversion/validation. The implementation computes the fractional target factor using bounded integer arithmetic and a cached table, avoiding platform-dependent floating-point exponentiation in the target-conversion step.

5.2 LWMA difficulty adjustment

The live difficulty calculation uses a linearly weighted moving average over the last 20 solve-time intervals. For interval i, solve time is the difference between consecutive block timestamps clamped to the configured range of 1 to 360 seconds. The work value for the interval is derived from the difficulty of the child block, using `2^(4D_i)`. Newer intervals receive a larger linear weight. The controller estimates elapsed seconds per unit of work, derives the work needed to reach the 60-second target, converts that work back to difficulty, and applies a symmetric per-block swing cap before enforcing the global floor and ceiling.

$$W_{time} = \Sigma(i \cdot clamp(\Delta t_i, 1, 360))$$

$$W_{work} = \Sigma(i \cdot 2^{(4D_i)})$$

$$time_per_work = W_{time} / W_{work}$$

$$next_work = 60 / time_per_work$$

$$D_{raw} = \log_2(next_work) / 4$$

$$D_{next} = clamp(D_{raw}, D_{parent} - \log_2(S)/4, D_{parent} + \log_2(S)/4)$$

Here `S` is the configured maximum multiplicative target swing (1.5 in the current configuration). The resulting difficulty is then clamped to the configured range [4.0, 63.0]. The first three chain blocks use the initial difficulty of 5.15. The block validator recomputes the required difficulty from local canonical history rather than trusting an incoming block's self-reported field.

5.3 Timestamp anti-manipulation

Median Time Past imposes a lower timestamp bound derived from recent block history, and the future-drift rule imposes an upper bound relative to the local clock. Per-interval solve-time clamping prevents a single extreme interval from dominating the weighted average. The controller's symmetric swing bound limits the maximum change in the target between adjacent blocks. Together, these measures constrain timestamp influence and smooth difficulty response while maintaining a 60-second target interval.

5.4 Mining workers and stale-candidate control

The mining engine prepares candidate blocks, invokes parallel or worker-based nonce search, monitors raw and session-mean hashrate, and pauses or rebuilds stale candidates. The candidate prewarm worker creates a candidate against an observed chain tip and publishes it only if the tip remains the expected parent. A stale candidate is discarded rather than mined on an obsolete parent. Before mining, the engine can check whether the local node is sufficiently synchronized with its best-known peer state and wait for synchronization subject to configured bounds. [8]

A candidate's timestamp remains fixed during the nonce search. That property keeps the time context used for state-root derivation aligned with the header eventually mined. The miner's VRF proof is generated using the current tip hash as alpha and passed into the header, while proof-of-work remains the block-production mechanism.

6. Stake-Weighted BFT Finality and Validator Safety

6.1 Consensus phases

Visold separates block production from finality. PoW miners construct and broadcast candidate blocks. After a block has passed validation and has been applied as the canonical block at its height, the node enters finalization handling. The validator set used to determine BFT eligibility is the snapshot associated with the start of that block's state transition. A `REGISTER` transaction contained in block N becomes effective for validator voting from block N+1; it cannot retroactively grant its sender voting rights over the block that registers it.

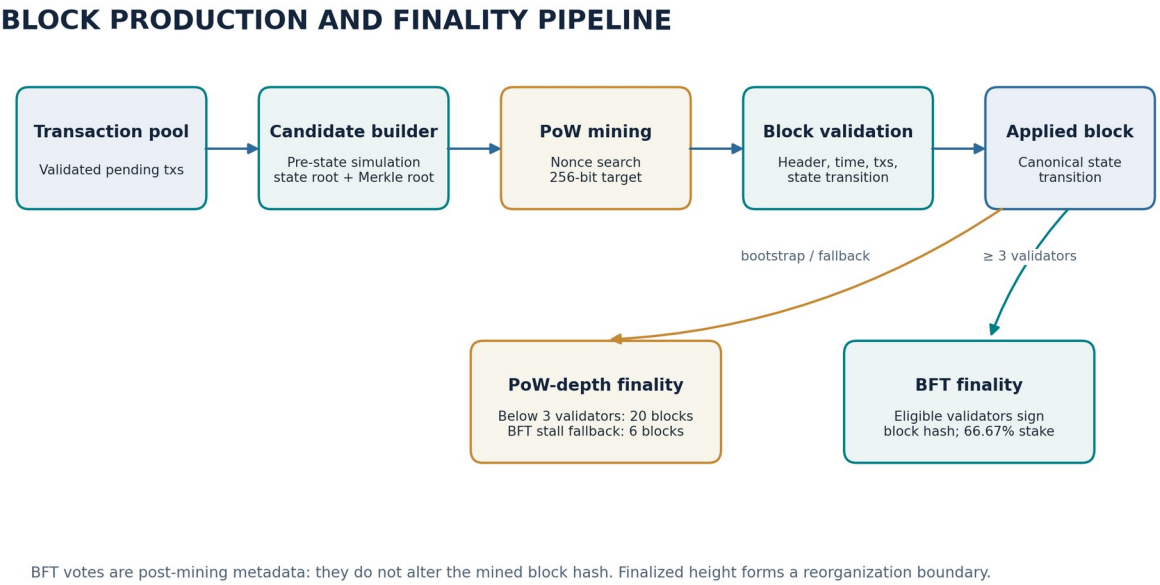


Figure 4. PoW block production, canonical application and finality decision paths.

6.2 Bootstrap PoW finality

BFT mode requires at least three registered investor/validator entries and a positive total eligible stake. Below that threshold, the chain remains in PoW bootstrap mode. Blocks are finalized by depth at 20 confirmations. This rule applies when the set contains zero, one or two validators; the presence of a partial validator set does not create an under-quorum BFT mode.

6.3 Stake-weighted BFT vote

When the validator set reaches the minimum count, an eligible validator signs the block hash with its application key and submits a vote containing validator address, signature and public key. The network gossips newly recorded votes. Receivers reconstruct the eligible validator set for the block height, require the claimed public key to derive the claimed validator address, verify the ECDSA signature over the block hash, and count each eligible validator address once. Quorum is weighted by stake, not by raw validator count.

```
approved_stake / total_eligible_stake >= 0.6667
```

```
Integer comparison: approved_stake_millionths × 1,000,000 >= 666,700 × total_stake
```

A block below quorum remains pending rather than being invalidated or discarded. As additional authenticated validator votes arrive, the same block can become BFT-finalized. The implementation stores the validator snapshot per block height so the denominator does not change when later role transactions modify the live table.

6.4 Vote authentication and equivocation

Validator-vote authentication checks the block hash, height-specific eligibility, public-key/address consistency and ECDSA signature. A validator may contribute at most one vote for one block hash at a given height. If the same validator signs a different block hash at the same height, the second signature is preserved as durable equivocation evidence but is excluded from quorum accounting. Vote history is stored independently of the canonical block table, so a subsequent reorganization does not erase the evidence needed to detect double-signing.

The configured malicious-vote slashing rate is 10% of the relevant stake. Evidence processing is separated from ordinary quorum counting: conflicting signatures are recorded, routed to equivocation handling, and must not increase the approved-stake numerator. This makes the finality certificate depend on authenticated, eligible and non-conflicting votes.

6.5 Finality fence and liveness fallback

BFT-finalized height is tracked as a reorganization fence. A reorganization cannot roll the canonical chain below the highest finalized block. If BFT participation stalls after the validator set is active, the liveness policy permits PoW-depth fallback after a timeout equal to the greater of the configured governance finality timeout and twice the six-block PoW finality depth. The fallback finalizes the block at the six-deep target; when validator participation resumes, later blocks can again use BFT finality.

Mode	Activation condition	Finality rule
PoW bootstrap	Fewer than 3 eligible validators	20-block confirmation depth
BFT active	At least 3 eligible validators and positive total stake	Authenticated votes >= 66.67% of the height-specific eligible stake
Degraded BFT liveness fallback	BFT active but finality stalled beyond configured timeout	Six-block PoW-depth finalization for the target block

6.6 Fork-choice and finalized history

Fork-choice evaluation uses canonical chain-work/difficulty history and finality constraints; the finalized-height fence is an independent safety boundary. A heavier candidate branch cannot be applied through a block at or below the finalized height when it would rewrite finalized history. Validator votes are not embedded in the PoW header hash, which keeps the mined block identifier stable while the finality certificate is accumulated asynchronously.

6.7 VRF and proposer-selection semantics

The implementation includes a deterministic VRF candidate-ranking helper that evaluates candidate keys against the latest block hash and selects the smallest output. This helper is distinct from the live PoW mining loop, which generates a tip-bound VRF proof as block metadata but continues to search for a valid PoW nonce. The protocol therefore does not describe validator stake or VRF output as replacing PoW block production in the current implementation. [6]

7. Monetary Policy, Fees and Reward Allocation

7.1 Denomination and initial issuance

The smallest accounting unit is one satoshi, equal to 10^-8 VSD. The genesis transaction is configured with a zero amount, so genesis does not create a premine. New supply enters through per-block subsidy. Transaction fees are transferred from transaction senders and redistributed with the block's reward pool; they are not counted as new issuance.

7.2 Subsidy schedule

The initial block subsidy is 1,000,000,000 satoshi (10 VSD). Every 20,000 blocks, the subsidy is multiplied by the exact rational factor 99/100. The computation uses integer exponentiation and explicit round-half-up arithmetic, then enforces a floor of 10,000,000 satoshi (0.1 VSD). This avoids floating-point exponentiation in the consensus reward path.

$$n = \text{floor}(\text{height} / 20{,}000)$$

$$\text{subsidy_sat}(\text{height}) = \max(10{,}000{,}000,\ \text{round_half_up}(1{,}000{,}000{,}000 \times (99/100)^n))$$

The schedule is not a fixed maximum-supply cap. Once the subsidy reaches the floor, each subsequent block continues to issue the floor amount. At the 60-second target interval, 52,560 blocks per 365-day year would correspond to approximately 5,256 VSD of floor-level annual subsidy; actual issuance follows the realized block count.

7.3 Reward distribution

The block distribution pool is the base subsidy plus fees collected from included transactions. The default allocations are 20% to the block proposer, 45% to the active-miner pool, and 35% to the validator pool. The active-miner pool is apportioned according to a consensus-safe miner activity score derived from canonical PoW contribution over a 100-block activity window. The validator pool is apportioned in proportion to registered stake. If no validators are registered, the 35% validator allocation is redirected to the miners, making the miner pool 80%. If both the miner and validator role tables are empty, the proposer receives the full pool.

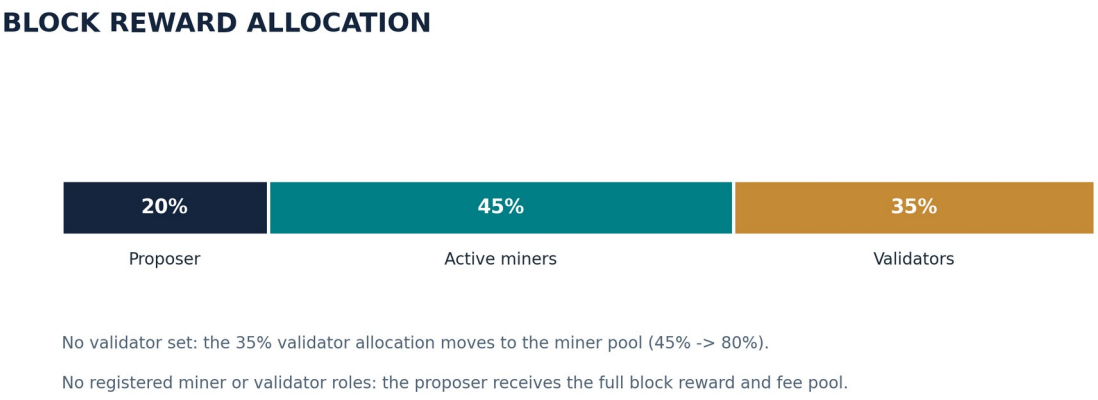


Figure 5. Configured reward distribution and bootstrap redirection rule.

All allocation arithmetic is carried out in integer satoshi. Ordered role records are read in deterministic address order. Per-recipient division remainders are assigned to the proposer; remaining integer dust is assigned to the burn address, keeping the credited total equal to the block pool. Reward computation uses the role set as it existed at the beginning of the block: registrations included in the current block affect subsequent blocks.

Miner-pool weights combine the miner registration score with recency-weighted canonical block production. For a 100-block window ending immediately before the rewarded block, a miner receives one activity point for a block at the newest edge, two for the preceding block, and so on up to 100 for the oldest edge of the window. The fixed-point score is multiplied by this activity credit; each miner receives the floor of its proportional share, and the integer remainder is assigned to the current proposer. A registered miner with no positive activity in the window receives no miner-pool share; if no registered miner has positive activity, the miner pool is assigned to the proposer.

$$\text{activity_a}(H,W) = \Sigma[h=H-W+1..H]\ 1\{\text{miner}(h)=a\} \cdot (W - (H-h))$$

$$\text{weight_a} = \max(\text{round}(\text{score_a} \cdot 10^6), 1) \cdot \text{activity_a}; \quad \text{share_a} = \text{floor}(M_{pool} \cdot \text{weight_a} / \Sigma_j \text{weight_j})$$

7.4 Registration thresholds and penalties

Role / rule	Configured value
Minimum investor/validator stake	200 VSD
Minimum miner registration stake	10 VSD
Equivocation slash rate	10%
BFT stake threshold	66.67%
Minimum BFT validator count	3
Miner activity lookback	100 canonical blocks
Rate-defection audit window	500 blocks
Rate-defection audit cadence	Every 50 blocks
Rate-defection trigger ratio	Actual / expected wins > 2.0
Minimum auditable miner share	3%
Rate-defection automatic slashing	Disabled by default; monitor/observe behavior is configured

7.5 Fee and reward separation

Fees are value redistribution rather than issuance. The block application path increments cumulative issued supply by the base subsidy only, then adds the transaction fee pool for reward allocation. This distinction is required for the conservation invariant: summing new issuance and reallocated existing balances must not double-count fees. [8]

8. Hashrate Governance and Miner Activity Accounting

8.1 Local hashrate governor

The hashrate governor maintains separate notions of raw hardware hashrate and the locally applied optimized mining rate. The required aggregate network rate is derived from difficulty and target block time. A node's actual hashrate is measured from local mining workers; it is not inferred from other peers. The local controller calculates a per-miner allowance and paces nonce work to keep a cooperative deployment near the target block interval.

Peer hashrate reports are signed, time-bounded observations. Fresh reports are used for a limited interval and then decay toward zero before expiry. These reports assist local pacing and dashboard estimates; they are not cryptographic proof of physical hashing and are not used as consensus reward weights. Consensus miner reward weights instead use canonical on-chain block contribution over a fixed recent window.

8.2 Hashrate reporting and weighted allocation

The configured share exponent is 0.5, so local optimization can apply a concave share transform rather than a strictly proportional allocation. Report freshness is 30 seconds, peer report TTL is 300 seconds, and the configured decay interval is 60 seconds. Reports are emitted every five seconds. This design allows a node to estimate network participation without allowing remote claims to become direct consensus facts.

8.3 Rate-defection audit

The rate-defection auditor compares a miner's observed canonical block wins against expected wins over a 500-block window. It only evaluates a miner whose estimated share is at least 3%, runs every 50 blocks, uses an actual-to-expected ratio threshold of 2.0, and requires three consecutive flagged windows. The configured automatic-slash switch is disabled by default, so the default mode records observations and advisory evidence rather than automatically applying a rate-defection penalty. This is distinct from the equivocation slashing path used for conflicting validator signatures. [9]

8.4 Consensus boundary

> **Consensus versus local policy**
> The local hashrate governor may slow a miner, but it cannot make a block invalid merely because the producer exceeded a locally estimated rate. Block validity is established by the PoW target, block structure, transaction checks, timestamp rules, state transition and consensus finality. Local raw H/s measurements and peer reports are not embedded in canonical reward calculations.

9. Peer-to-Peer Networking and Topology Management

9.1 Network services

The network context contains TCP and UDP transports, peer-connection and session management, TLS, message framing, compression, discovery, reputation, Kademlia routing, peer exchange (PEX), topology policy, NAT traversal, relay services and SPV support. The default P2P port is 8338 and is used for both TCP and UDP. The local JSON-RPC listener uses the selected P2P port plus one (8339 by default) and binds to loopback. An optional ICE relay uses a nearby available TCP port while avoiding the RPC port.

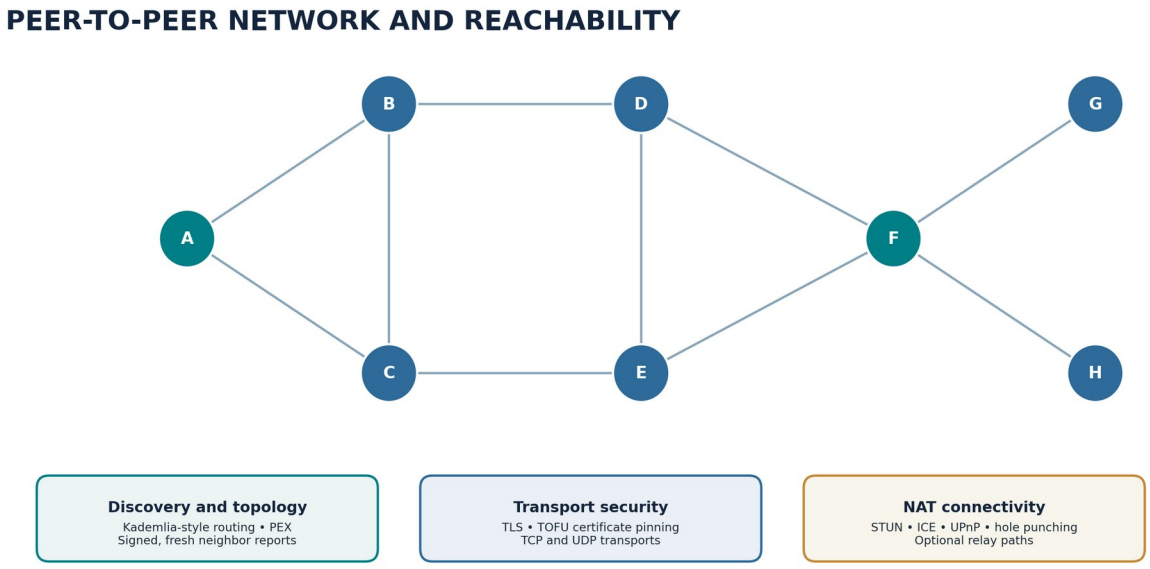


Figure 6. Peer connectivity is managed through authenticated sessions, discovery and transport-adaptation services.

9.2 Message framing and compression

The TCP protocol uses newline-delimited JSON message frames. Larger payloads can be compressed; the compression subsystem prefers zstd when available and provides a zlib fallback. The current configuration enables compression for payloads above 512 bytes. A framing marker identifies compressed content so receivers can select the proper decoder. Decompression paths are bounded to limit memory amplification from hostile or malformed messages.

9.3 TLS and peer admission

TLS protects peer connections. Each node uses a self-generated P-256 certificate and TOFU fingerprint pinning. The peer handshake includes application-level identity and software/logic compatibility information. New connections are subject to handshake timeouts, a cap on concurrently pending TLS handshakes, rate controls and a proof-of-work-style admission puzzle. The configured puzzle has a 19-bit base difficulty, with temporary additional difficulty during high connection bursts from a network prefix.

9.4 Peer discovery and routing

Peer discovery combines known peers, seed information, PEX and Kademlia-style routing. The Kademlia context uses a 160-bit identifier space and a configured bucket size of 20. PEX is periodically exchanged, gossip messages have a bounded hop lifetime, and peer reputation influences connection selection and exclusion. The ordinary-node hard peer cap is 30. A soft topology target of three peers is used by the adaptive topology policy; it is a routing target rather than a hard consensus rule or a guarantee that a node will always have exactly three live connections.

9.5 Signed adaptive topology reports

Topology reports describe node identity, neighbor identifiers, degree, completeness, seed status, issue time and sequence number. Reports are signed with the node application key and accepted only when the signature, identity binding, freshness, sequence and structural constraints are valid. Candidate ranking prefers peers with lower reported degree and rotates among ties. Stale or missing reports are treated conservatively. Automatic edge pruning requires fresh, complete and mutually consistent reports; asymmetric adjacency claims are not accepted as paths for proving redundancy. This protects the overlay from treating one-sided or stale claims as safe connectivity.

9.6 UDP transport

The UDP transport is divided into wire encoding, session state, reliability and latency measurement. It supports session identifiers, sequencing/acknowledgment behavior, bounded retransmission and stale-session recovery. The TCP path remains available for reliable peer communication and synchronization; UDP can provide an alternate transport under network conditions where its reachability and reliability layer are useful.

9.7 NAT traversal and relaying

The NAT subsystem integrates STUN mapping discovery, ICE-style connectivity checks, UPnP mapping, UDP hole punching and optional relay support. A STUN-observed endpoint is an observation of a mapping, not proof that arbitrary inbound peers can reach it. Direct connectivity depends on firewall and NAT behavior. A relay must itself be reachable by the relevant peers; placing a relay behind the same restrictive NAT does not automatically solve inbound reachability.

9.8 Message integrity, quotas and peer isolation

Peer connections have bounded handshake durations, message sizes, bandwidth allowances, per-peer quotas and connection counts. Rate-limit responses can be correlated with request identifiers and can include retry guidance, allowing clients to back off without treating temporary server capacity as a permanent protocol failure. P2P synchronization uses separate request correlation and per-peer recovery paths so one unresponsive peer does not monopolize the whole synchronization operation.

9.9 RPC isolation

The JSON-RPC endpoint is intentionally local-only. Node-to-node P2P connectivity does not require public RPC exposure. A remote operator can use a secure tunnel or separately authenticated access gateway, but the protocol's normal P2P listeners and local RPC listener remain separate trust boundaries. [10, 23]

10. Storage Architecture, Pruning and Authenticated Synchronization

10.1 Storage backend profiles

The storage layer supports a lightweight SQLite profile and a higher-throughput PGX profile composed of PostgreSQL, RocksDB and Redis. In PGX mode, PostgreSQL is the canonical store for account/role and transaction state, RocksDB stores block bodies, headers and associated key-value indexes, and Redis provides a cache. The implementation maintains an atomic block-application boundary and a canonical-tip marker so a database commit and the RocksDB block record can be reconciled after interrupted writes. If PGX dependencies are not requested or unavailable, the default configuration uses SQLite and the selected local key-value block database adapter.

Profile	Components	Role
Default lightweight	SQLite + configured local block DB adapter	Portable local persistence for consumer-device deployments.
PGX	PostgreSQL + RocksDB + Redis	Separated canonical state, block storage/indexing and cache for larger deployments.

10.2 Single-writer and atomic block application

A canonical block is applied as one logical state transition. The storage layer serializes SQLite access under a re-entrant API lock and wraps block application in an atomic transaction. In PGX mode, a thread-local PostgreSQL connection holds the block transaction while consensus-state writes and reads participate in the same transaction; RocksDB writes are paired with rollback/reconciliation markers. Readers do not accept an orphan RocksDB tip as canonical unless the durable canonical-tip marker confirms the same height and hash.

10.3 Rolling block-body pruning

Rolling pruning is enabled by default and keeps a recent body window of 600 blocks. Older block bodies can be compacted into slim header records while canonical history metadata remains available. Pruning runs periodically and in bounded batches to limit foreground impact. Selective archival behavior can retain additional block data based on configured transaction/account criteria. The result is a node storage policy that separates the ability to follow and validate the header chain from indefinite retention of every historical transaction body.

10.4 State snapshots

The snapshot engine captures the complete state at a canonical anchor height. Snapshot creation is dispatched to a background thread and captures state under the chain writer lock so that the state payload and its block height cannot describe different tips. The payload includes the anchor height, block hash, state root and the state components needed for restoration. It is serialized deterministically, compressed with zlib, stored under a height-specific key and indexed by a manifest containing anchor metadata, size and digest. The default interval is 600 blocks and the default retention count is three snapshots.

10.5 Authenticated bootstrap snapshots

When rolling pruning is enabled, the node generates the periodic bootstrap anchors required for new participants even if the optional general-purpose snapshot setting is disabled. At each 600-block bootstrap interval, a deterministic state snapshot is tied to a canonical block anchor. The following coinbase transaction carries the commitment that allows a joining node to authenticate the snapshot against the canonical header and transaction commitment path. A node can validate compact headers from genesis to the anchor, verify the commitment, restore the corresponding state and then continue normal synchronization after the anchor.

10.6 Chunked fast synchronization

Fast synchronization is available for chains at or above height 600. Snapshot transfer uses 1 MiB chunks, permits up to eight parallel peers, applies a 30-second timeout per chunk and allows three retries. The default maximum uncompressed snapshot size is 256 MiB. The default local-anchor requirement ensures that a downloaded snapshot is accepted only after the receiving node has established the required canonical header anchor. Bounded decompression and digest validation protect the restore process from malformed or oversized payloads. [11, 22]

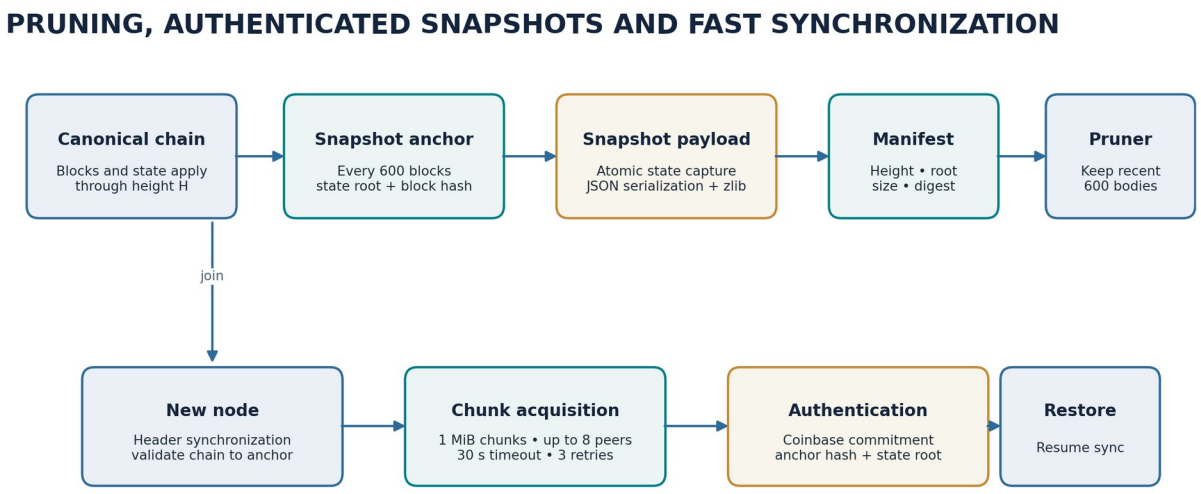


Figure 7. Snapshot creation, pruning and authenticated fast-sync lifecycle.

10.7 Historical data and index separation

The storage model distinguishes canonical headers, block bodies, transaction lookup metadata, account state, contract code and contract storage. Pruning block bodies does not imply that every associated index or state datum is deleted. SPV clients and explorers can use header and Merkle proof interfaces where full-body retrieval is not required. Historical block-body availability remains a node retention property, while the consensus chain identity continues to be anchored by the header chain and state commitments.

11. Visold Virtual Machine (VVM)

11.1 Runtime purpose and instruction model

The Visold Virtual Machine provides deterministic execution for deployable contracts and contract calls. Its instruction set is organized around arithmetic, comparisons, bitwise operations, SHA-3-style hashing, environment access, account/contract inspection, memory, persistent storage, control flow, stack manipulation, logs, calls, creation, revert/return, precompiles, time-lock conditions and native payment-channel behavior. The source defines 206 opcode-related integer constants, including five type constants, corresponding to 201 executable instruction identifiers.

11.2 Execution frame

A VVM execution frame carries the program counter, operand stack, memory, private registers, calldata, return-data context, gas state, message-call environment and contract context. Eight private registers supplement the conventional operand stack. Typed-value facilities make execution type distinctions explicit. Each nested call is bounded by a maximum call depth of 1,024; each frame has a maximum memory allocation of 1 MiB and the operand stack is bounded at 1,024 entries.

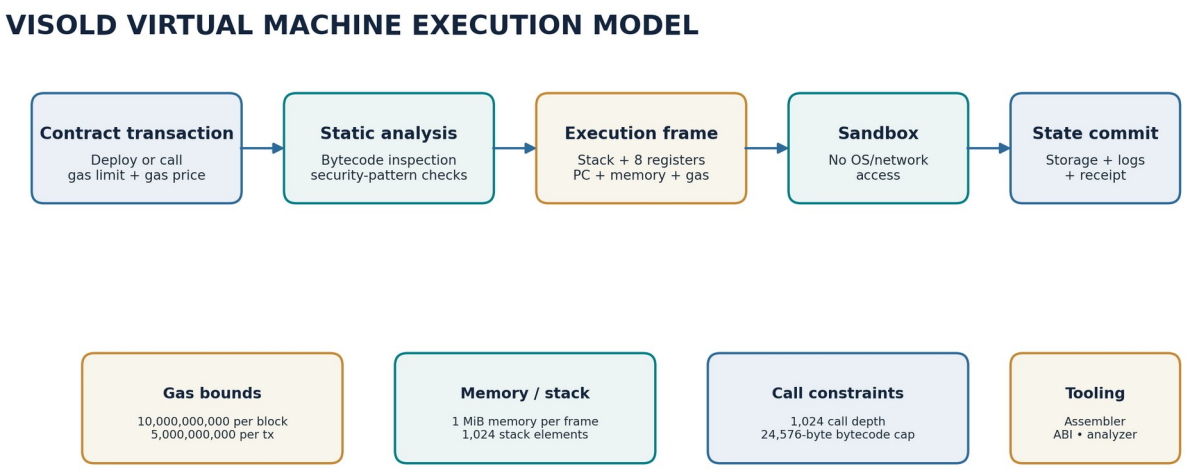


Figure 8. VVM transaction admission, analysis and bounded execution model.

11.3 Gas and resource bounds

Parameter	Configured value
Block gas limit	10,000,000,000 gas
Per-transaction gas cap	5,000,000,000 gas
Minimum gas price	1 × 10^-8 VSD per gas unit
Maximum call depth	1,024
Maximum memory per frame	1,048,576 bytes
Maximum stack depth	1,024
Maximum deployed bytecode	24,576 bytes
Base contract deployment gas	32,000
Bytecode deployment cost	200 gas per byte

Gas is charged according to opcode and data-operation costs. The block gas cap bounds aggregate execution work in a block, while transaction-level gas limits bound individual calls. The memory, stack, call-depth and bytecode ceilings provide

independent controls against resource-exhaustion programs. Gas price is committed in satoshi-per-gas units in the canonical transaction encoding.

11.4 Contract deployment and invocation

A deploy transaction carries contract initialization data and a contract name/address context. The VVM deployment path validates transaction structure, enforces bytecode size, invokes static analysis and executes initialization within a bounded frame. A call transaction identifies the target contract, calldata and execution budget. The runtime resolves code, constructs the execution context, meters operations and applies the resulting storage writes through the canonical state engine. Simulation and gas estimation interfaces allow callers to execute against a non-committing state view.

11.5 Environment and control instructions

VVM environment instructions expose controlled blockchain context such as caller, origin, call value, block number, timestamp, difficulty, gas limit, chain identifier, coinbase and contract balance. Memory operations use explicit offsets and byte/word semantics. Persistent storage uses key-value slot operations. Control-flow instructions use validated jump destinations. Contract-to-contract call families include ordinary calls and static calls, with call context and return-data handling bounded by the frame and gas model.

11.6 Time-locks and payment channels

Time-lock instructions express conditions tied to block height or timestamp windows, including after/before/window-style predicates. The VM also contains native payment-channel instructions and state-channel signature handling, allowing a contract or channel program to express channel-specific payment and settlement logic. These facilities are integrated into the opcode/runtime model rather than requiring every application to reproduce the same primitives in ordinary bytecode.

11.7 Static bytecode analysis

The VVM static analyzer scans bytecode before deployment for common risk patterns. The implementation includes checks for reentrancy-style call-before-state-write flows, suspicious integer/stack patterns, time-lock misuse and payment-channel instruction relationships. Static analysis is a developer-facing detection layer; it is not a formal proof that arbitrary contract code is free of vulnerabilities. The runtime still enforces gas, memory, stack, call-depth and state-transition rules during execution.

11.8 Developer tooling and receipts [12]

The VVM context includes an assembler, ABI encoder/decoder, event indexer, precompile interface, static analyzer, contract naming/address derivation and RPC methods for deployment, calls, simulation, gas estimation, code retrieval and event queries. Receipts provide an execution record for the transaction path and its resulting state changes. These tools make the virtual machine inspectable by both wallet applications and external development tooling.

12. Layer-2 Rollup Architecture

12.1 Design model

Visold's Layer-2 subsystem maintains an account state and state tree for off-chain transfers while using Layer 1 as the settlement and data-availability layer. An L2 transaction is signed by the user and binds the chain ID under a dedicated `VSD-L2` domain tag. This prevents a signature created for the Layer-1 transaction format from being reused as an L2 transfer authorization.

12.2 L2 transaction format

An L2 transfer contains sender, receiver, integer amount in satoshi, per-sender nonce, timestamp, signature and public key. Its canonical signing string includes the chain ID and a fixed L2 domain prefix. The transaction identifier hashes the signing bytes together with signature and public key. L2 transactions do not carry the full Layer-1 fee, memo, transaction type, gas limit, gas price or expiry fields; batch settlement fees are handled by the sequencer's L1 rollup submission.

12.3 State tree and batch lifecycle

The L2 state comprises accounts, balances and nonces committed into a state-tree root. The sequencer collects validated L2 transactions, applies them in order to derive a new root, seals a batch when it reaches 2,048 transactions or a soft age threshold of 30 seconds, canonicalizes the batch transaction rows and compresses the data using zlib. The compressed data

is retained on Layer 1 for data availability. A single compressed submission is structurally capped at 1,000,000 bytes; decompression is bounded to 32 MiB.

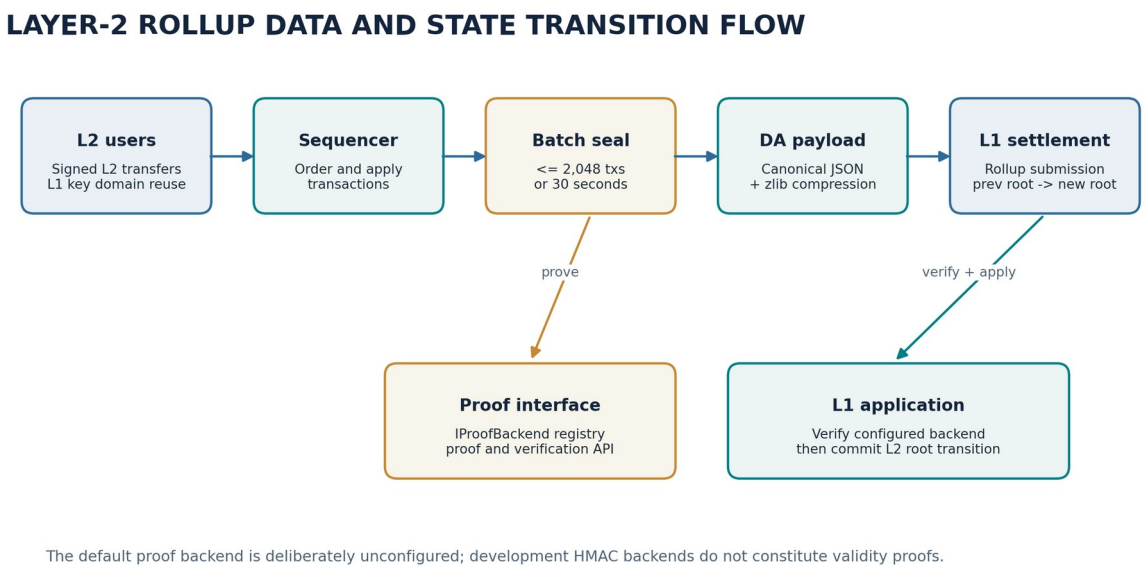


Figure 9. Layer-2 transaction batching, compressed data availability and Layer-1 settlement.

12.4 Rollup submission

A rollup submission contains a monotonically increasing batch identifier, previous L2 root, new L2 root, proof bytes, compressed canonical transaction data, backend name and backend-security descriptor. The submission is serialized as compact, sorted JSON and embedded in an L1 transaction of type `rollup`. At L1 application time, the verifier precompile checks the configured backend and the claimed state transition before the Layer-2 state tree commits the new root. The L2 subsystem retains 128 past roots to support rollback and reorganization handling.

12.5 Proof backend interface and production semantics

The proof framework defines an `IProofBackend` contract and a registry. The shipped local-development backend uses HMAC-SHA256 to authenticate a claim over the previous root, new root and batch hash; this is an authentication mechanism, not a zero-knowledge validity proof. The simulated backend maps to the same development-security class. The subprocess SNARK backend defines an integration contract for external prover and verifier executables but requires the operator to supply those binaries and configure the audited state-transition proof system. The default configured backend is `no-backend-configured`; when a deployment is explicitly configured in mainnet mode, the registry rejects backends marked as development, HMAC, simulated or stub and prevents sequencer startup with an unsafe backend.

Layer-2 proof semantics
The rollup data model deliberately separates data availability from validity verification. The compressed canonical batch permits re-execution and inspection; the proof backend authenticates or proves the transition according to its declared security class. A development HMAC must never be described as a SNARK or as a cryptographic proof of arbitrary state-transition correctness.

12.6 Deposits and withdrawals

The L2 state model includes bridge and withdrawal sentinel addresses and L1/L2 deposit and withdrawal operations. These operations map value between the Layer-1 account state and Layer-2 account tree. Receiver validation recognizes protocol sentinel addresses and contract addresses, while ordinary L2 transfers reject self-send operations and require positive integer amounts. L2 balance sufficiency and per-account sequencing are evaluated by the L2 state transition engine. [13]

13. Mempool Policy and Commit-Reveal Transaction Ordering

13.1 Pending transaction admission

The mempool maintains pending transactions before block inclusion. Admission checks include transaction parsing, signature and sender binding, expiry, nonce, balance/fee affordability, type-specific validation and local rate/dust controls. The local 1,000-satoshi minimum fee floor is an anti-flood admission policy; consensus block validation uses the transaction's canonical fee and state transition rules. Candidate construction can select a subset of pending transactions and computes a dry-run state root so that the block header commits to the state produced by the chosen candidate body.

13.2 Commit-reveal mechanism

The optional MEV protection path uses a commitment transaction followed by a reveal. The commitment is a zero-value self-transfer whose commitment data binds a salted hash. The reveal supplies the salt and a Merkle path linking the commitment to the included block. The configured policy uses a three-block reveal delay and a maximum age of 20 blocks. The protection switch is disabled by default and can be enabled through the node policy/configuration surface. When activated, it changes the permitted transaction-ordering workflow; it does not alter the ordinary transaction signature domain.

13.3 Scope of protection

Commit-reveal reduces information leakage from a transaction before the reveal stage, making straightforward front-running harder. It does not hide the eventual revealed transaction, eliminate miner/validator coordination, guarantee fair ordering under all network topologies or replace application-level slippage and deadline controls. It should therefore be understood as an optional ordering-protection mechanism rather than a universal MEV solution. [14]

14. Self-Healing, Resilience and Operational Safeguards

14.1 Monitoring pipeline

The Self-Healing Blockchain System (SHBS) observes block-application activity through a monitor bus. Monitors collect transaction, gas, validator and contract-interaction signals. A statistical detector evaluates observations against learned baselines and produces anomaly reports with categories, severity and confidence. A decision engine applies thresholds, signal confirmation and suppression rules. A healing-action layer validates and executes selected responses.

14.2 Graded response model

Response class	Representative behavior	Scope
Observe	Record metrics, anomaly records and evidence in the action log.	Node-local monitoring state.
Rate control	Apply per-address rate limits through a sliding window.	Local admission and execution policy.
Temporary freeze	Freeze an address or contract for a bounded duration.	Local to the node; not a consensus rule.
Validator alert	Send an evidence-bearing alert to connected validators.	Network notification; not a state transition.
Rollback proposal	Preview and coordinate a state rollback through validator voting and explicit confirmation gates.	Governance/validator-controlled recovery path.

14.3 Local versus consensus behavior

SHBS freezes are node-local protective controls. A locally frozen account is excluded by that node's candidate-building policy, but the freeze registry is not consulted by canonical block validation and is not itself a consensus state rule. Nodes can therefore continue to validate a valid block even if their local anomaly monitor has frozen a sender for candidate selection. This separation prevents different local monitoring decisions from creating different block-validity outcomes.

14.4 Rollback controls

The rollback subsystem records state snapshots and prepares a preview of the effect of reversing recent blocks. Critical rollback operations are routed through validator voting and explicit confirmation stages in the hardened executor path. The SHBS RPC interface exposes status, logs, validator vote submission, unfreeze controls and rollback dry-run/confirmation calls. The design treats rollback as an exceptional recovery operation rather than ordinary fork choice.

14.5 Resilience components

The resilience context includes a panic circuit breaker, hardened-core overlays, invariant checks and a sentinel monitor. Safety invariants include account-supply conservation and consistency between canonical issuance and balances. The sentinel watches a designated primary node through local RPC health and chain-height progression; it alerts the operator and can indicate standby state, but it does not take over a validator signing key or autonomously cast votes. Recovery authority and validator key control remain explicit operator responsibilities.

14.6 Operational observability

Metrics and RPC status surfaces expose block/peer health, invariant outcomes, database sizes, circuit-breaker state, sentinel state, SHBS action history and mining telemetry. This makes the response system inspectable and allows operators to distinguish a protocol-level rejection from a node-local operational action. [15]

15. Protocol Governance and Version Upgrades

15.1 Software and protocol versioning

The software release identifier and protocol-version number are separate. Consensus-sensitive changes require deterministic activation because two nodes interpreting the same block under different consensus rules may diverge. The governance engine represents an upgrade proposal as a state machine with DORMANT, SIGNALING, LOCKED_IN, ACTIVE and FAILED phases.

15.2 Signaling and activation lifecycle

Phase	Semantics
DORMANT	Proposal is announced; signaling window has not begun.
SIGNALING	Miners identify the proposed protocol version in block metadata; the engine counts the share of signaling blocks.
LOCKED_IN	The configured threshold is reached; the activation height is fixed.
ACTIVE	The activation height is reached and the new rule set is enforced.
FAILED	The proposal is rolled back or emergency-disabled.

The configured support threshold is 75% of blocks in the signaling window. Activation is delayed from lock-in by a configured number of blocks. Every node can derive proposal phase and activation height from the same chain history, avoiding an external coordinator as the source of truth for upgrade state.

15.3 Emergency disable and signed authorization

The emergency-disable vote preimage is domain-separated and length-prefixed. It binds the genesis hash, protocol version, operator address, timestamp and a nonce. The signature therefore cannot be reused for another chain, version, signer identity, moment or nonce. Governance endpoints expose protocol version, upgrade status, proposal and emergency-disable operations.

15.4 Activation-height discipline

Consensus constants such as transaction format activation, Merkle construction, reward allocation and finality rules must not be edited in place after a chain has produced history. Such changes require an activation-height protocol upgrade so historical blocks remain interpreted under the rules that originally validated them. The upgrade mechanism is therefore central to forward compatibility and controlled consensus evolution. [16]

16. Wallet, Identity, RPC and Operator Interfaces

16.1 Wallet and identity system

The wallet creates and stores application keys, signs Layer-1 transactions, derives address identities and supports recovery-phrase workflows. The identity context provides a decentralized name registry. Identity claims are encoded through register transaction conventions and associated with the address that submitted the claim. Names are therefore resolved from the chain's registration state rather than from a centralized naming authority.

16.2 Role registration

The role registry records miners and investors/validators using register transactions and stake thresholds. A miner role requires at least 10 VSD; an investor role requires at least 200 VSD. Role changes are state transitions and take effect according to block application order. The active role set used for reward distribution is the set persisted before the current block's registration transactions are processed.

16.3 JSON-RPC interface

The node exposes a JSON-RPC 2.0 server over HTTP on the local loopback interface. The default RPC port is one greater than the configured P2P port. The server uses an operator-local bearer token. The RPC surface is organized around chain and transaction queries, peer information, governance and protocol state, safety diagnostics, contract deployment/calls/simulation, SHBS controls and Layer-2 operations. RPC access is deliberately separated from public P2P connectivity.

16.4 CLI and terminal UI

The same entry script supports an interactive terminal interface, command-line wallet and node operations, version output and an embedded test entry point. The terminal interface adapts to narrow screen widths and includes network/port settings, status panels and node controls. A Termux helper can manage a detached tmux session, while Android process and power-management policies remain operating-system concerns.

16.5 Multi-instance data isolation

Each concurrently running node on one device must use a distinct `VISOLD_DATA_DIR`, because the data directory contains the database, wallet, keystore, peers, configuration, TLS certificate and node-status files. Different P2P ports alone do not isolate node state. This design permits multiple independent node instances on a single Android device while preserving wallet/database separation. [17, 23]

17. End-to-End Node Lifecycle and Protocol Interactions

17.1 Startup

1. The process resolves the data directory and loads configuration, protocol constants, storage settings and network port policy.
2. The storage layer opens the selected local or PGX storage profile, initializes schemas and reconciles the canonical-tip marker.
3. The node loads or initializes the wallet and application identity, encrypted keystore and TLS certificate identity.
4. The node composes chain, state, consensus, mempool, mining, network, governance, VVM, rollup, SHBS, sentinel, RPC and CLI services.
5. P2P listeners and peer discovery become active under the configured TCP/UDP port plan; RPC remains loopback-only.
6. The node synchronizes headers, blocks or authenticated snapshots as appropriate, then mining safety gates permit candidate production according to synchronization status.

17.2 Transaction-to-finality sequence

7. A wallet constructs a transaction using the chain ID, canonical fields, account nonce, expiry and any type-specific data.
8. The wallet signs the canonical payload. The public key, signature and tx identifier accompany the transaction.
9. Peers exchange the transaction; receiving nodes validate its format, sender binding, signature, expiry, nonce, fee and local mempool policy.
10. A miner builds a candidate body, performs candidate-state simulation, computes the Merkle root and resulting state root, and constructs a header bound to the parent block.

11. The miner searches for a nonce satisfying the difficulty-derived 256-bit target and broadcasts the mined block.
12. Each node independently validates the block, applies the state transition and records the canonical block atomically.
13. If the eligible pre-state validator set contains at least three validators, authenticated stake-weighted votes are accumulated until the 66.67% threshold is reached; otherwise PoW-depth finality applies.

17.3 Reorganization and recovery

A competing chain is evaluated against chain and finality constraints. Difficulty calculations are invalidated when canonical history changes. The state engine and storage layer roll back uncommitted or non-finalized transitions according to the chain's reorganization policy. The highest finalized height remains a hard fence. Snapshot recovery is used for a node whose retained block bodies do not cover the required history; a snapshot is accepted only after the anchor header chain and bootstrap commitment have been authenticated.

18. Verification Architecture and Multi-Node Test Domains

18.1 Test organization

The public repository contains 105 `tests/test_*.py` modules plus embedded verification harnesses under `visold/testing`. The test inventory is organized around protocol behavior and regression families rather than a single monolithic test file. These test modules cover consensus finality, validator vote authentication, reorganization fences, PoW and difficulty behavior, transaction/state application, P2P connectivity, peer admission, TCP and UDP transport, synchronization, pruning, snapshot authentication, rollup plumbing, VVM rollback, storage atomicity, RPC behavior, and terminal input/rendering. [18]

18.2 Multi-peer and network integration domains

Domain	Representative source tests
Topology and mesh behavior	`test_adaptive_peer_topology_policy.py`; `test_p2p_mesh_admission_regressions_20261020.py`; `test_mesh_block_relay_dedup_20261021.py`
Connection lifecycle and retry	`test_p2p_liveness_and_handshake_handoff_20261022.py`; `test_peer_retry_cooldown_fixes_20261021.py`; `test_tcp_peer_decay_stale_session_race_20261022.py`
TCP and UDP recovery	`test_tcp_eof_diagnostics_20261022.py`; `test_tcp_recv_blockingio_retry_20261022.py`; `test_udp_receiver_self_heal_20261027.py`; `test_udp_stale_reconnect_regression_20261027.py`
Snapshot/pruned-node synchronization	`test_pruned_reconnect_authenticated_bootstrap.py`; `test_pruned_resync_integration.py`; `test_two_node_pruning_boundary_reconnect.py`; `test_snapshot_relay_rollback_failover_20261024.py`
Consensus and finality	`test_consensus_finality_security_fix_20261028.py`; `test_consensus_finality_audit_regressions_20261028.py`; `test_consensus_hardening_regressions.py`
Storage and atomicity	`test_sqlite_atomic_begin_race.py`; `test_snapshot_prune_failure_atomicity_20261028.py`; `test_pgx_rollback_cleanup.py`
VVM and rollup	`test_vvm_rollback_regressions.py`; `test_vvm_selfdestruct_regressions.py`; `test_snark_backend_plumbing_20261024.py`

18.3 Verification layers

- Unit-level checks: canonical encodings, monetary conversions, hashing, signatures, address derivation, DAA formulas, Merkle trees, opcode behavior, compression bounds and snapshot serialization.
- State-transition checks: balance conservation, nonce order, role activation heights, fee/subsidy accounting, contract rollback, state-root equality and failed-application atomicity.
- Consensus checks: validator snapshots, vote eligibility, signature authentication, quorum arithmetic, double-sign evidence, finality promotion, fallback and reorganization fencing.
- Network integration checks: multi-peer admission, reciprocal topology, block gossip, duplicate suppression, peer retries, stale-session recovery, transport framing, compression, rate limits and NAT/network failure handling.

- Synchronization checks: bootstrap headers, chunk transfer, snapshot digest/anchor authentication, pruned-history recovery, reconnect from offline state, rollback boundary safety and resumed block application.
- Operational checks: command-line flows, RPC request handling, terminal rendering, Android/Termux launch and isolated multi-instance data directories.

18.4 Reproducibility package

A reproducibility package for a journal or independent engineering review can consist of the public source repository, a pinned commit identifier, test commands, configuration profiles, raw runtime logs, benchmark scripts and machine descriptions. Consensus-sensitive benchmark records should identify the software commit, configuration, chain height, node count, hardware profile, network conditions and the metric definition. Multi-node results should distinguish several node processes sharing one host from nodes running on physically independent devices, because their CPU, memory, network and storage contention models differ.

18.5 Selected Multi-Node and Recovery Evidence

The public repository preserves dated validation records from the Visold development lineage. The examples below are attributed to the specific dates and baselines stated in those records. They provide concrete integration evidence for network topology, mining liveness, pruned-history recovery and authenticated snapshot synchronization.

18.5.1 Seven-to-eight-node live topology observation

The 2026-10-22 P2P EOF validation report records 91 focused regression tests passing and a live node-count expansion from seven to eight. During a 300-second observation containing 296 topology samples, all eight nodes reported seven peers each, the mesh contained 28 symmetric physical links, and session identifiers remained stable. The report records no BlockingIOError, peer-send failure, transport-abort or peer-disconnected event during that observation. The test report identifies the then-current source identity as a 149-file development baseline; this is a historical network-validation record, not a re-labelling of the 152-file package fingerprint documented in this whitepaper. [19]

18.5.2 Three-process mining and liveness smoke test

The 2026-10-27 liveness/mining report records a targeted combined regression suite of 59 passing tests. In a controlled run of three local node processes lasting approximately 30 seconds, one mining node and two non-mining peers all reached height 7, each node reported two active peers, and all observed circuit breakers remained CLOSED. The report explicitly distinguishes this short integration run from longer unattended-device observations; the result is useful evidence of short-run connectivity and chain progress under the recorded configuration. [20]

18.5.3 Authenticated recovery after block-body pruning

The 2026-10-19 pruning-recovery report describes an end-to-end regression using the real Visold Blockchain, SQLite storage, block validation/application, pruning, authenticated header verification, snapshot restore, mempool reconciliation and receipt-proof verification. The source advanced to height 20 and pruned the body containing a previously pending transfer. The receiver recovered through an authenticated snapshot at height 18 and full blocks 19–20; source and receiver canonical tip hashes and state roots matched, the receiver balance matched at 5 VSD, the confirmed transaction was removed from persistent and in-memory mempool state, and the receipt proof was verified. The report notes that only the P2P transport was mocked for this particular integration scenario; separate repository tests exercise mailbox routing. [21]

18.5.4 Two-node authenticated snapshot synchronization

The 2026-10-24 synchronization record documents an isolated two-node synchronization in which a fresh node advanced from height 0 to source height 1,256. Both nodes ended at the same canonical tip hash: `000072f11b0647d3e0d60b40431746e11bdab73d4b62890226eb4a64f891c6a4`. The associated protocol behavior uses authenticated snapshot anchors and paced header-page requests so that a pruned serving node need not provide every historical block body. [22]

19. Technical Contributions and Design Significance

19.1 Integrated consensus separation

The most consequential architectural choice is the separation of PoW block production from stake-weighted finality. PoW determines who constructs a candidate block through a target-constrained search; BFT validator votes determine when an applied block crosses the finality threshold. A pre-state validator snapshot makes voting rights height-specific, while persistent vote history makes equivocation detectable even if a voted block is later orphaned. The three-validator activation requirement prevents a one- or two-validator set from being mistaken for a functioning BFT quorum.

19.2 Consumer-oriented networking and storage

The networking and storage layers treat reachability, connection degree, storage retention and synchronization cost as first-class node-engineering concerns. The adaptive topology layer uses signed and freshness-checked neighbor reports; peer selection considers reported degree; edge pruning requires reciprocal evidence and alternate-path safety. The storage layer retains headers while pruning older bodies, creates complete state snapshots and authenticates bootstrap anchors through the chain. Together these features reduce the need for every node to retain and serve every historical body while preserving a verifiable route back to canonical state.

19.3 Canonical state and execution boundaries

Canonical transaction encoding makes field boundaries explicit, binds the chain ID, and commits gas and monetary values in satoshi units. The state engine centralizes state mutation. The VVM imposes explicit resource bounds and exposes contract tooling, while static analysis provides an early warning layer for selected bytecode patterns. The Layer-2 framework reuses the account-key namespace but separates L2 transaction authorization by domain tag and carries compressed canonical data plus state-root transitions to Layer 1.

19.4 Local policy does not become consensus

A recurring engineering contribution is the explicit boundary between local optimization and consensus. Examples include local hashrate throttling versus canonical miner contribution rewards, local mempool admission floor versus block validity, local SHBS freezes versus deterministic state validation, candidate block size tuning versus the fixed block maximum, and peer topology suggestions versus consensus rules. This boundary is essential in a decentralized system because local resource conditions and operator policy cannot be assumed identical across nodes.

19.5 Research value

The implementation creates an engineering case study for integrating conventional PoW, weighted BFT finality, resource-aware node operation, snapshot-based state bootstrap and native contract execution in a single Python package. Its research value is strongest when examined through measurable questions: how validator participation affects finality latency; how the 20-block bootstrap depth behaves under measured hash-rate conditions; how topology policy changes connectivity and message overhead; how snapshot size and synchronization time scale with state cardinality; how pruning alters local disk growth; and how contract gas/memory bounds shape execution cost. These questions can be evaluated against the public implementation and reproducible deployment profiles.

19.6 Design principles for protocol evolution

Future protocol changes should preserve the same separation of concerns: consensus constants require activation-height governance; storage schema changes require atomic migration and recovery; network policy changes must not redefine block validity; optional cryptographic acceleration must preserve byte-level signatures; and proof-system integrations must explicitly declare their security class. This provides a basis for extending the protocol without silently changing the semantics of historical blocks.

Appendix A. Consensus and Deployment Parameter Reference

Parameter	Value	Classification / explanation
CHAIN_ID (repository default)	vsd-testnet-1	Repository default is `vsd-testnet-1`; a future mainnet must use a distinct chain ID and coordinated genesis.
TARGET_BLOCK_TIME	60 s	Consensus DAA target.
INITIAL_DIFFICULTY	5.15	Initial difficulty during early bootstrap.
MIN_DIFFICULTY	4.0	Global DAA floor.
MAX_DIFFICULTY	63.0	Global DAA ceiling.
DIFF_LWMA_WINDOW	20	Solve-time intervals in the LWMA window.
DIFF_LWMA_MIN_TS_STEP	1 s	Per-interval solve-time lower clamp.

Parameter	Value	Classification / explanation
DIFF_LWMA_MAX_TS_STEP	360 s	Per-interval solve-time upper clamp.
DIFF_LWMA_MAX_SWING	1.5	Maximum multiplicative target swing per block.
DIFF_MTP_WINDOW	11	Recent blocks used for median time past.
MAX_FUTURE_DRIFT	7,200 s	Maximum future timestamp allowance.
MAX_BLOCK_SIZE	4,096,000 bytes	Consensus block body cap.
MIN_BLOCK_SIZE	1 MiB	Candidate-builder lower cap; not a consensus minimum.
TX_FEE_RATE	1%	Normal transfer-fee rate.
MIN_TX_FEE_SAT	1,000 sat	Mempool admission policy floor.
INITIAL_REWARD	10 VSD	Initial block subsidy.
MIN_REWARD	0.1 VSD	Subsidy floor.
REWARD_DECAY_BLOCKS	20,000	Subsidy decay interval.
REWARD_DECAY_NUM/DEN	99/100	Exact decay factor.
REWARD_CREATOR	20%	Block proposer allocation.
REWARD_ALL_MINERS	45%	Active miner allocation when validator pool exists.
REWARD_ALL_VALIDATORS	35%	Stake-weighted validator allocation.
MIN_INVESTOR_STAKE	200 VSD	Minimum investor/validator registration stake.
MIN_MINER_STAKE	10 VSD	Minimum miner registration stake.
MIN_VALIDATORS_FOR_BFT	3	BFT activation count.
BFT_THRESHOLD_MILLIONTHS	666,700	66.67% stake quorum.
POW_FINALITY_DEPTH	6	PoW depth fallback after BFT activation.
BOOTSTRAP_POW_FINALITY_DEPTH	20	PoW depth while BFT set has fewer than 3 validators.
SLASH_RATE	10%	Configured validator equivocation penalty.
MINER_ACTIVITY_WINDOW	100 blocks	Canonical contribution lookback.
AUTO_SLASH_RATE_DEFECTION	False	Rate-defection monitor starts in observe-only mode.
DEFAULT_PORT	8338	P2P TCP/UDP default.
RPC_PORT_OFFSET	+1	Local-only JSON-RPC listener.
MAX_PEERS	30	Ordinary-node hard connection cap.
TOPOLOGY_TARGET_PEERS	3	Adaptive topology soft target.
TLS_CERT_VALIDITY_DAYS	90	Self-signed P-256 certificate rotation interval.
ROLLING_PRUNE_WINDOW	600 blocks	Recent body retention window.
SNAPSHOT_INTERVAL	600 blocks	General snapshot interval.
SNAPSHOT_KEEP	3	Snapshot retention count.
SNAPSHOT_CHUNK_SIZE	1 MiB	Fast-sync transfer chunk size.
SNAPSHOT_MAX_PARALLEL_PEERS	8	Snapshot chunk peer concurrency.
SNAPSHOT_CHUNK_TIMEOUT	30 s	Chunk request timeout.
SNAPSHOT_CHUNK_MAX_RETRIES	3	Chunk retry limit.
SNAPSHOT_MAX_UNCOMPRESSED_BYTES	256 MiB	Snapshot decompression ceiling.
L2_MAX_BATCH_SIZE	2,048 tx	Maximum L2 transactions per batch.
L2_BATCH_MAX_AGE_SECS	30 s	Soft maximum batch age.
L2_ROOT_HISTORY_DEPTH	128	L2 root history retained for rollback.

Parameter	Value	Classification / explanation
L2_PROOF_BACKEND	no-backend-configured	Default safe backend-selection state.
VVM_BLOCK_GAS_LIMIT	10,000,000,000	Aggregate VVM gas cap per block.
VVM_TX_GAS_CAP	5,000,000,000	Per-transaction VVM gas cap.
VVM_MAX_MEMORY_BYTES	1 MiB	Memory bound per frame.
VVM_MAX_STACK_DEPTH	1,024	Operand stack bound.
VVM_MAX_CALL_DEPTH	1,024	Nested call bound.
VVM_MAX_BYTECODE_SIZE	24,576 bytes	Deployed bytecode ceiling.

Appendix B. Data Structures and Message Concepts

Object	Key fields / contents
Transaction	Version, sender, receiver, amount, fee, timestamp, public key, signature, tx_id, memo, nonce, expiry, type, data, gas limit, gas price, contract name.
Block header	Version, protocol version, height, parent hash, timestamp, miner, difficulty, nonce, Merkle root, state root, VRF proof and output.
Validator vote	Block hash, validator address, signature, public key; eligibility resolved from the block-height validator snapshot.
Topology report	Version, node identifier, public key, sorted neighbors, degree, completeness, seed flag, issue timestamp, sequence and signature.
Snapshot manifest entry	Height, state root, compressed size, block hash and bootstrap digest.
Rollup submission	Batch ID, previous/new L2 roots, proof bytes, compressed data-availability blob, backend name and backend-security descriptor.
L2 transaction	Sender, receiver, amount_sat, nonce, timestamp, signature, compressed public key and l2_tx_id.

Appendix C. Source-Code Map

Protocol area	Principal modules
Configuration and shared types	`visold/kernel/config.py`; `kernel/units.py`; `kernel/serialization.py`; `kernel/messages.py`
Cryptography and wallet	`visold/crypto/ecc.py`; `crypto/hashing.py`; `crypto/vrf.py`; `crypto/keystore.py`; `crypto/mnemonic.py`; `visold/wallet/`
Transactions and blocks	`visold/ledger/transaction.py`; `ledger/block.py`; `ledger/addressing.py`
Consensus and mining	`visold/consensus/difficulty.py`; `consensus/hashrate_governor.py`; `consensus/slashing.py`; `visold/chain/consensus_engine.py`; `visold/mining/`
Canonical state and chain	`visold/state/engine.py`; `visold/chain/blockchain.py`; `visold/chain/roles.py`
Storage and snapshots	`visold/storage/storage.py`; `storage/backends.py`; `storage/block_database.py`; `storage/snapshot.py`; pruning modules
P2P and topology	`visold/network/p2p.py`; `peer_connection.py`; `topology_policy.py`; `kademlia.py`; `tls.py`; `udp/`; `nat/`; `bootstrap_sync.py`; `block_download.py`
VVM	`visold/vm/opcodes.py`; `vm/engine.py`; `vm/frame.py`; `vm/assembler.py`; `vm/abi.py`; `vm/precompiles.py`; `vm/static_analyzer.py`

Protocol area	Principal modules
Layer 2	`visold/rollup/l2_state.py`; `rollup/batches.py`; `rollup/proofs.py`; `rollup/sequencer.py`; `rollup/compact_codec.py`
MEV and pending transactions	`visold/mempool/pool.py`; `mempool/mev.py`
Resilience and monitoring	`visold/selfhealing/`; `visold/resilience/`
Governance and identity	`visold/governance/`; `visold/identity/`
Node and interfaces	`visold/node/`; `visold/api/rpc_server.py`; `visold/cli/`
Tests	`tests/` and `visold/testing/`

Appendix D. Glossary

Term	Definition
BFT	Byzantine fault tolerant; a protocol family designed to reach agreement despite a bounded set of faulty or malicious participants.
Canonical chain	The single chain history currently accepted by the node under fork-choice and finality constraints.
Candidate block	A locally constructed block awaiting proof-of-work and network validation.
Finality fence	The highest finalized height below which chain reorganization is prohibited.
LWMA	Linearly weighted moving average; a controller giving greater weight to more recent intervals.
MTP	Median Time Past; a median timestamp constraint applied to recent blocks.
PGX	Visold storage profile combining PostgreSQL, RocksDB and Redis.
PoS	Proof of stake; here, stake determines validator vote weight for BFT finality.
PoW	Proof of work; nonce search against a difficulty-derived hash target.
TOFU	Trust on first use; a peer certificate fingerprint is pinned after initial observation.
VSD	Visold native asset and chain ticker.
VVM	Visold Virtual Machine, the deterministic contract execution runtime.
State root	Cryptographic commitment to the resulting canonical account/contract/role state.
Rollup	Layer-2 batch of transactions settled by a Layer-1 commitment and configured proof backend.

GitHub Repository References

[1] Visold Project. Public source-code repository and project record. Repository root.

[2] Visold Project. Project overview, protocol identity, entry-point commands and repository layout. README.md.

[3] Visold Project. Package build-identity record; SHA-256 fingerprint and source-file count. visold/_build_identity.json.

[4] Visold Project. Shared protocol configuration, units, serialization and dependency-layer foundations. visold/kernel/config.py; visold/kernel/units.py; visold/kernel/serialization.py.

[5] Visold Project. Canonical transaction, block, addressing and state-transition implementation. visold/ledger/transaction.py; visold/ledger/block.py; visold/state/engine.py.

[6] Visold Project. Proof-of-work difficulty control, consensus engine, validator finality and slashing logic. visold/consensus/difficulty.py; visold/chain/consensus_engine.py; visold/consensus/slashing.py.

[7] Visold Project. Cryptographic primitives, signature verification, key handling and VRF implementation. visold/crypto/ecc.py; visold/crypto/hashing.py; visold/crypto/keystore.py; visold/crypto/vrf.py.

[8] Visold Project. Monetary issuance, subsidy schedule, fee calculation and reward allocation. visold/economics/rewards.py; visold/consensus/hashrate_governor.py; visold/mining/engine.py.

[9] Visold Project. Hashrate reporting, activity accounting and rate-defection monitoring. visold/consensus/hashrate_reports.py; visold/consensus/rate_defection.py.

[10] Visold Project. Peer-to-peer networking, peer sessions, adaptive topology, UDP and NAT traversal. visold/network/p2p.py; visold/network/peer_connection.py; visold/network/topology_policy.py; visold/network/udp/transport.py; visold/network/nat/ice.py.

[11] Visold Project. Persistent storage, block database, pruning and state snapshot implementation. visold/storage/storage.py; visold/storage/block_database.py; visold/storage/snapshot.py; visold/storage/rolling_window_pruner.py; visold/storage/state_pruner.py.

[12] Visold Project. Virtual-machine instruction set, execution engine, assembler, precompiles and static analysis. visold/vm/opcodes.py; visold/vm/engine.py; visold/vm/assembler.py; visold/vm/precompiles.py; visold/vm/static_analyzer.py.

[13] Visold Project. Layer-2 account state, rollup batches, proof backend and sequencer implementation. visold/rollup/l2_state.py; visold/rollup/batches.py; visold/rollup/proofs.py; visold/chain/sequencer.py.

[14] Visold Project. Pending-transaction pool and commit-reveal / MEV protection implementation. visold/mempool/pool.py; visold/mempool/mev.py.

[15] Visold Project. Self-healing monitors, anomaly detection, response orchestration and runtime invariants. visold/selfhealing/orchestrator.py; visold/selfhealing/detection.py; visold/resilience/safety_invariants.py; visold/resilience/panic_breaker.py.

[16] Visold Project. Protocol versioning, upgrade governance and identity registry. visold/governance/engine.py; visold/governance/versioning.py; visold/identity/names.py.

[17] Visold Project. Node composition, wallet, RPC service and operator interfaces. visold/node/visold_node.py; visold/wallet/wallet.py; visold/api/rpc_server.py; visold/cli/entry.py.

[18] Visold Project. Automated test suite and selected consensus, networking, topology and recovery regression tests. tests directory; consensus finality security tests; adaptive peer topology tests; authenticated pruning recovery tests.

[19] Visold Project. P2P EOF validation report, including the dated seven-to-eight-node topology observation. P2P_EOF_TEST_RESULTS_2026-10-22.md.

[20] Visold Project. Liveness and mining telemetry fix report and controlled three-process node run. LIVENESS_MINING_FIX_REPORT_2026-10-27.md.

[21] Visold Project. Authenticated recovery after pruned-block synchronization rejection. PRUNED_RECONNECT_RECOVERY_FIX_REPORT_2026-10-19.md.

[22] Visold Project. Current authenticated snapshot and pruned-node synchronization behavior. docs/CURRENT_SYNC_BEHAVIOR_2026-10-24.md.

[23] Visold Project. Deployment, network-port behavior, Android/Termux operation and cryptographic acceleration notes. docs/NETWORK_PORTS.md; docs/ANDROID_TERMUX_GUIDE.md; docs/CRYPTO_ACCELERATION.md.

[24] Visold Project. Software license and accompanying notices. docs/LICENSE.txt.

Author and Source Attribution

Himadri Maity is identified as the author of this technical whitepaper. Correspondence: maity23himadri@gmail.com. The sole source repository for the Visold software described in this document is the public GitHub repository: https://github.com/maity23himadri/Visold_blockchain. The software license is the MIT License as published in docs/LICENSE.txt. This whitepaper is a technical description of that software and does not alter or replace the software license.

Suggested citation: Maity, H. (2026). Visold (VSD): A Modular Proof-of-Work Blockchain with Stake-Weighted BFT Finality, Resource-Aware Networking, and a Native Virtual Machine. Technical Whitepaper, Version 1.3, October 30, 2026. Source repository: https://github.com/maity23himadri/Visold_blockchain